

Desenvolvimento de Aplicações Móveis

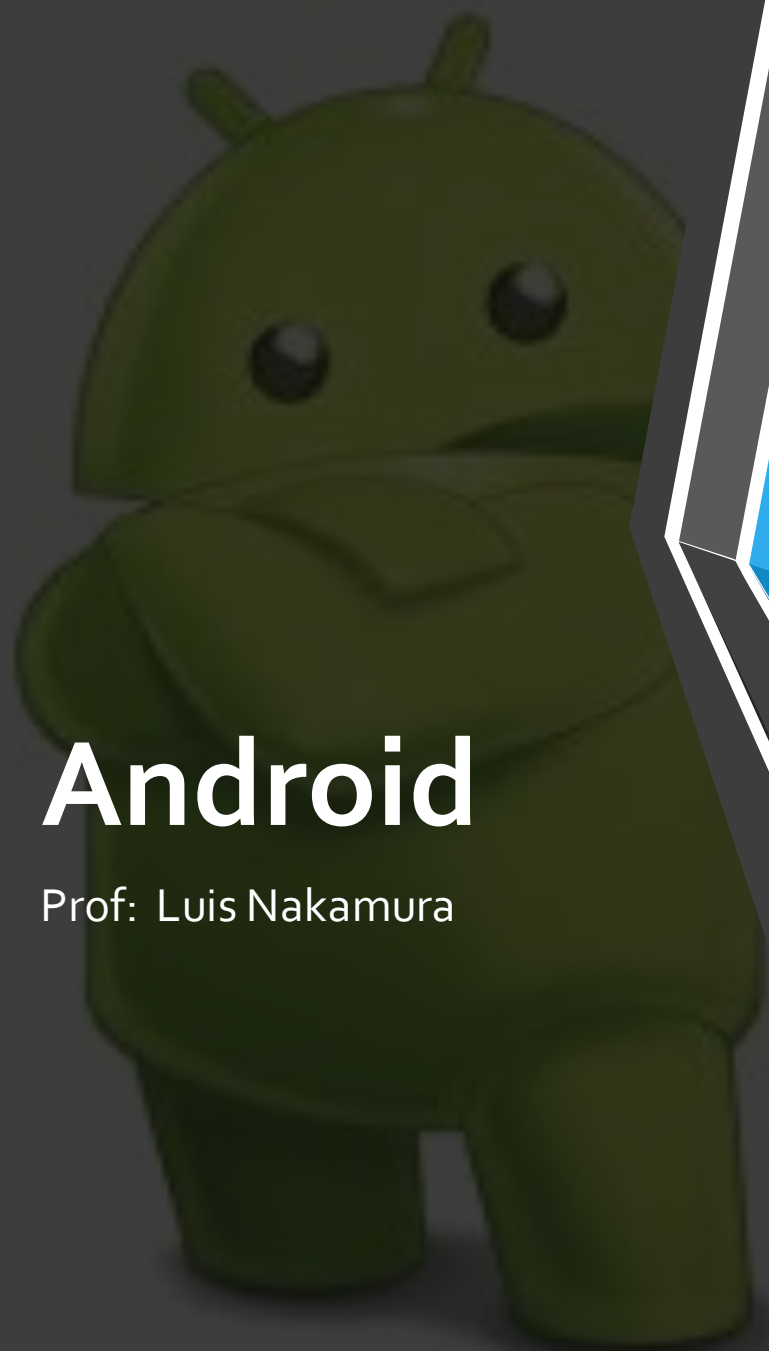
Prof: Luis Nakamura



Permissões (Resumido)

Apresentação da Aula: Permissões

- **Objetivos da Aula:**
 - Teoria sobre Permissões (básico)
 - Prática: Solicitar uma permissão no aplicativo.



Android

Prof: Luis Nakamura



Teoria: Permissões

- **Permissões no Android:**
 - As permissões permitem garantir a privacidade do usuário protegendo o seguinte:
 - **Dados restritos**, como o estado do sistema e os dados de contato de um usuário.
 - **Ações restritas**, como conectar-se a um dispositivo pareado e gravar áudio

<https://developer.android.com/guide/topics/permissions/>
<https://developer.android.com/reference/android/Manifest.permission>

Teoria: Permissões

- **Permissões no Android:**
 - A partir do **Android 6 (Marshmallow – API 23)** o usuário pode conceder a permissão de acesso para um determinado recurso acessando as configurações do sistema ou dinamicamente quando o acesso ao recurso é solicitado pela aplicação. Um exemplo é quando queremos utilizar o armazenamento de arquivos externos (no cartão SD).

<https://developer.android.com/guide/topics/permissions/>
<https://developer.android.com/reference/android/Manifest.permission>

Teoria: Permissões

- **Permissões no Android:**
 - **A partir da versão 6**, conceder as permissões apenas no arquivo `AndroidManifest.xml` não terá efeito prático.
 - **Cabe ao desenvolvedor (via programação)** fazer a verificação se existe a permissão de uso do recurso e, se necessário, requisitar as permissões ao usuário.

<https://developer.android.com/guide/topics/permissions/>
<https://developer.android.com/reference/android/Manifest.permission>

Teoria: Permissões

- Permissões no Android:

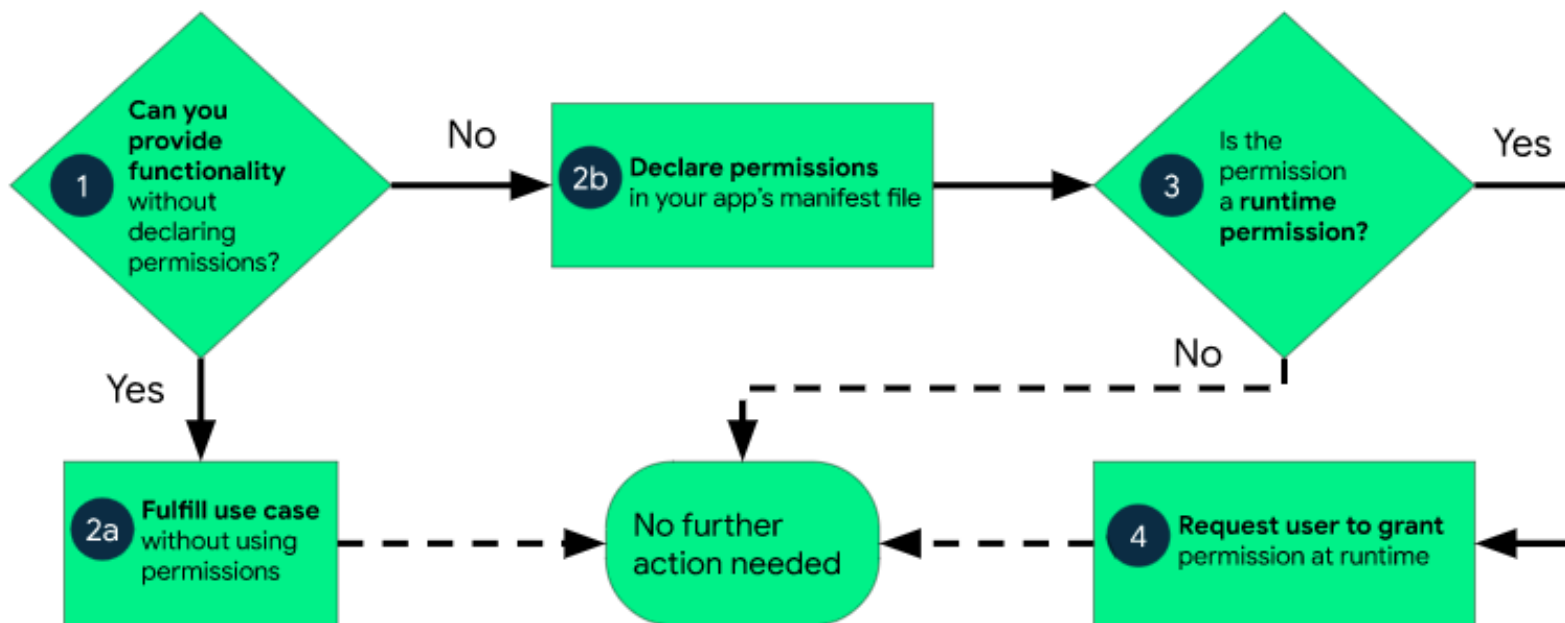


Figura 1. Diagrama que mostra o fluxo de trabalho de alto nível para usar permissões no Android.

<https://developer.android.com/guide/topics/permissions/>
<https://developer.android.com/reference/android/Manifest.permission>

Teoria: Permissões

- Permissões no Android:

Permissões de tempo de instalação

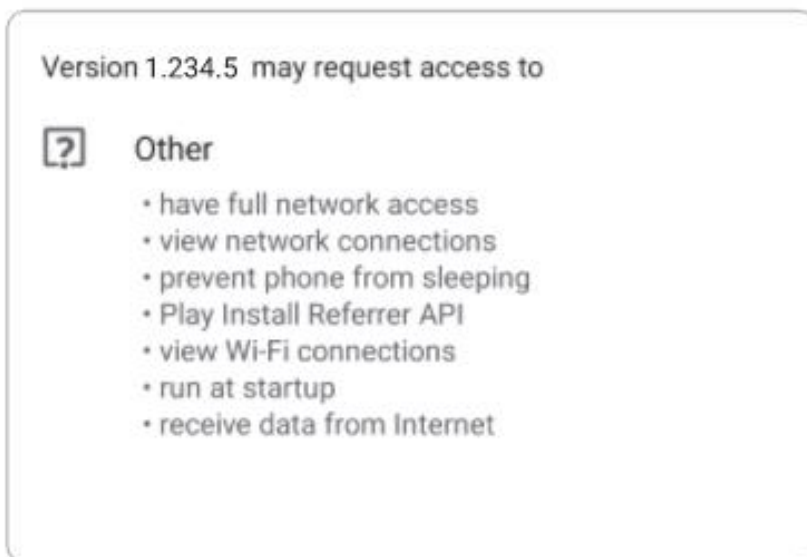


Figura 2. A lista de permissões de tempo de instalação de um app, que aparece em uma app store.

Permissões de execução

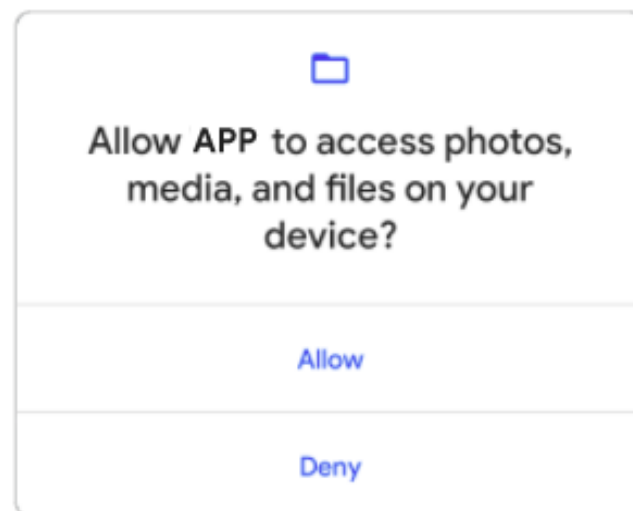


Figura 3. A solicitação de permissão do sistema exibida quando o app solicita uma permissão de execução.

<https://developer.android.com/guide/topics/permissions/>
<https://developer.android.com/reference/android/Manifest.permission>

Teoria: Permissões

- Permissões no Android:

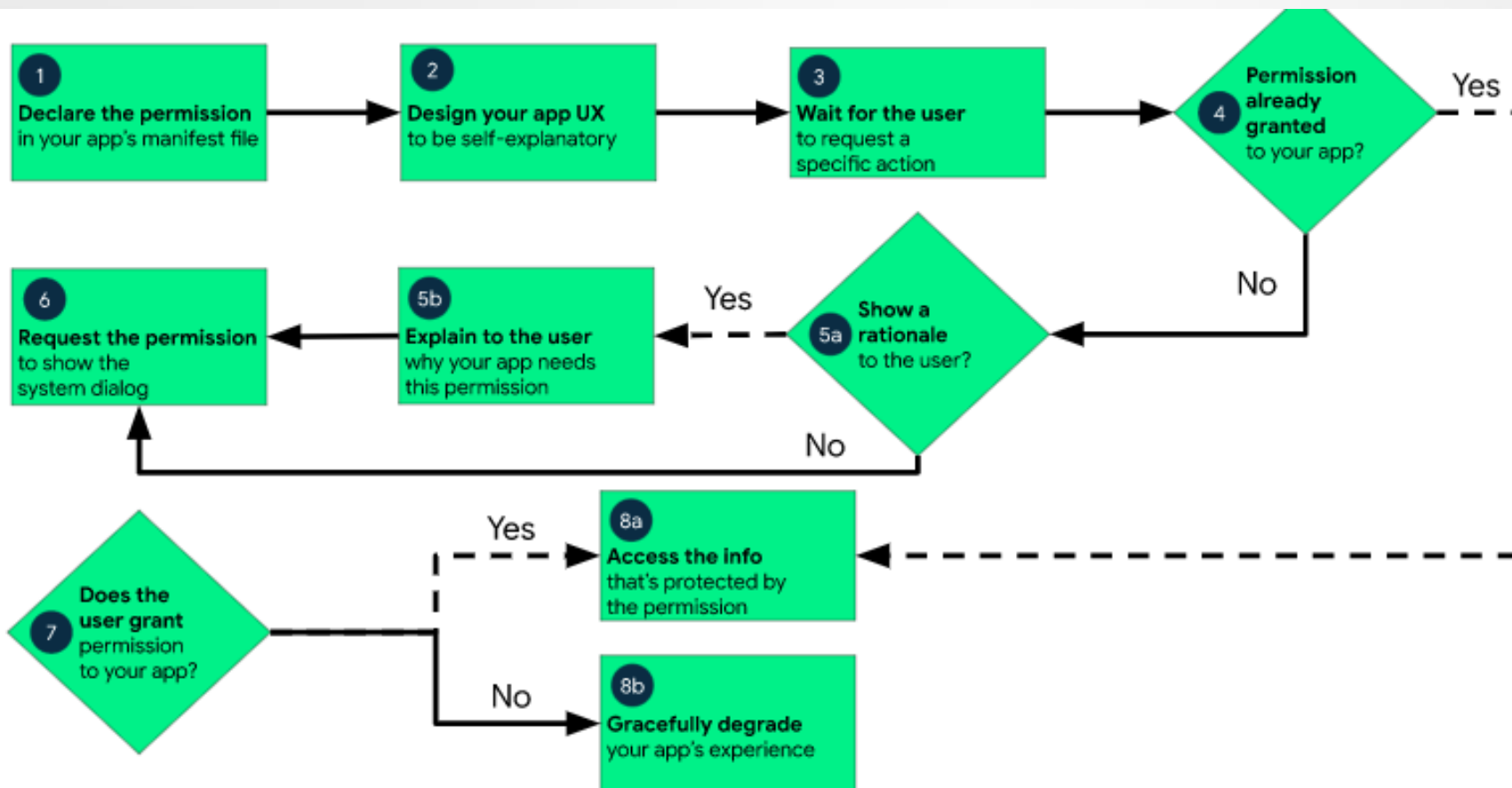


Figura 1. Diagrama que mostra o fluxo de trabalho para declarar e solicitar permissões de execução no Android.

<https://developer.android.com/guide/topics/permissions/>

<https://developer.android.com/reference/android/Manifest.permission>

Prática 1: Permissões

- Método para verificar a permissão de acesso (Glauber, 2015):

```
public static final int PERMISSAO_SDCARD = 0;
```

```
private boolean checarPermissao(String permissao, int requestCode) {  
    if (ActivityCompat.checkSelfPermission(this, permissao)  
        != PackageManager.PERMISSION_GRANTED) {  
        if (ActivityCompat.shouldShowRequestPermissionsRationale(this, permissao))  
        {  
            Toast.makeText(this, "Você tem que habilitar essa permissão para poder salvar o arquivo",  
                           Toast.LENGTH_SHORT).show();  
        }  
        ActivityCompat.requestPermissions(this, new String[]{permissao}, requestCode);  
        return false;  
    }  
    return true;  
}
```

Prática 1: Permissões

- Método para verificar a permissão de acesso (Glauber, 2015):

```
@Override
public void onRequestPermissionsResult(int requestCode, String[] permissions, int[] grantResults) {
    super.onRequestPermissionsResult(requestCode, permissions, grantResults);

    switch (requestCode) {
        case PERMISSAO_SDCARD: {
            if (grantResults[0] == PackageManager.PERMISSION_GRANTED) {
                Toast.makeText(this, "Permissão concedida", Toast.LENGTH_SHORT).show();
            }
            else {
                Toast.makeText(this, "Permissão negada", Toast.LENGTH_SHORT).show();
            }
        }
    }
}
```

Prática 1: Permissões

- Método para verificar a permissão de acesso (Glauber, 2015):

```
private void salvarNoSdCard() {  
    boolean temPermissao = checarPermissao(  
        Manifest.permission.WRITE_EXTERNAL_STORAGE, PERMISSAO_SDCARD);  
  
    if(!temPermissao) {  
        return;  
    }  
  
}
```

Prática 1: Permissões

- Outro Exemplo:

```
final static int MY_PERMISSION_REQ_WRT_EXT_STRG = 777;
public boolean getPermissionAPI23()
{
    int permit = -1;

    if (Build.VERSION.SDK_INT >= 23) {
        permit = ContextCompat.checkSelfPermission(this,
            Manifest.permission.WRITE_EXTERNAL_STORAGE);
    }
    else {
        permit = PermissionChecker.checkSelfPermission(this, Manifest.permission.WRITE_EXTERNAL_STORAGE);
    }

    if( permit != PermissionChecker.PERMISSION_GRANTED) {

        ActivityCompat.requestPermissions(this,
            new String[]{Manifest.permission.WRITE_EXTERNAL_STORAGE},
            MY_PERMISSION_REQ_WRT_EXT_STRG);

        return false;
    }
    else
    {
        return true;
    }
}
```

<https://developer.android.com/training/permissions/requesting.html?hl=pt-br>

Prática 1: Permissões

- Outro Exemplo:

```
@Override
public void onRequestPermissionsResult(int requestCode,
    String permissions[], int[] grantResults) {
    switch (requestCode) {
        case MY_PERMISSION_REQ_WRT_EXT_STRG: {
            // grantResults será maior que zero se o usuário aceitou
            if (grantResults.length > 0
                && grantResults[0] == PackageManager.PERMISSION_GRANTED) {

                //Continua o código...
            }
            return;
        }
    }
}
```

Prática 2: Permissões

- Outro Exemplo (Google):

```
// Register the permissions callback, which handles the user's response to the
// system permissions dialog. Save the return value, an instance of
// ActivityResultLauncher, as an instance variable.
private ActivityResultLauncher<String> requestPermissionLauncher =
    registerForActivityResult(new RequestPermission(), isGranted -> {
        if (isGranted) {
            // Permission is granted. Continue the action or workflow in your
            // app.
        } else {
            // Explain to the user that the feature is unavailable because the
            // features requires a permission that the user has denied. At the
            // same time, respect the user's decision. Don't link to system
            // settings in an effort to convince the user to change their
            // decision.
        }
    });
```

Prática 2: Permissões

- Outro Exemplo (Google):

```
if (ContextCompat.checkSelfPermission(  
    CONTEXT ✎, Manifest.permission.REQUESTED_PERMISSION ✎) ==  
    PackageManager.PERMISSION_GRANTED) {  
    // You can use the API that requires the permission.  
    performAction(...);  
} else if (shouldShowRequestPermissionRationale(...)) {  
    // In an educational UI, explain to the user why your app requires this  
    // permission for a specific feature to behave as expected. In this UI,  
    // include a "cancel" or "no thanks" button that allows the user to  
    // continue using your app without granting the permission.  
    showInContextUI(...);  
} else {  
    // You can directly ask for the permission.  
    // The registered ActivityResultCallback gets the result of this request.  
    requestPermissionLauncher.launch(  
        Manifest.permission.REQUESTED_PERMISSION ✎);  
}
```


Prática 2: Permissões

- Outro Exemplo (Google 2): Gerenciar o código de solicitação de permissão.

```
if (ContextCompat.checkSelfPermission(  
    CONTEXT ✎, Manifest.permission.REQUESTED_PERMISSION ✎) ==  
    PackageManager.PERMISSION_GRANTED) {  
    // You can use the API that requires the permission.  
    performAction(...);  
} else if (shouldShowRequestPermissionRationale(...)) {  
    // In an educational UI, explain to the user why your app requires this  
    // permission for a specific feature to behave as expected. In this UI,  
    // include a "cancel" or "no thanks" button that allows the user to  
    // continue using your app without granting the permission.  
    showInContextUI(...);  
} else {  
    // You can directly ask for the permission.  
    requestPermissions(CONTEXT ✎,  
        new String[] { Manifest.permission.REQUESTED_PERMISSION ✎ },  
        REQUEST_CODE ✎);  
}
```

Prática 2: Permissões

- Outro Exemplo (Google 2): Gerenciar o código de solicitação de permissão.

```
@Override
public void onRequestPermissionsResult(int requestCode, String[] permissions,
    int[] grantResults) {
    switch (requestCode) {
        case PERMISSION_REQUEST_CODE:
            // If request is cancelled, the result arrays are empty.
            if (grantResults.length > 0 &&
                grantResults[0] == PackageManager.PERMISSION_GRANTED) {
                // Permission is granted. Continue the action or workflow
                // in your app.
            } else {
                // Explain to the user that the feature is unavailable because
                // the features requires a permission that the user has denied.
                // At the same time, respect the user's decision. Don't link to
                // system settings in an effort to convince the user to change
                // their decision.
            }
            return;
        // Other 'case' lines to check for other
        // permissions this app might request.
    }
}
```

Prática: Permissões

- Permissão no Arquivo AndroidManifest.xml (Sempre!)

```
<manifest ... >
    <uses-permission android:name="android.permission.SEND_SMS"/>
    ...
</manifest>
```

- Permissões para Localização:

```
<manifest ... >
    <!-- Always include this permission -->
    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />

    <!-- Include only if your app benefits from precise location access. -->
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
</manifest>
```

Prática: Permissões

- Localização em primeiro plano
 - Recomendado/Obrigatório: Um serviço em primeiro plano.

```
<!-- Recommended for Android 9 (API level 28) and lower. -->  
<!-- Required for Android 10 (API level 29) and higher. -->  
<service  
    android:name="MyNavigationService"  
    android:foregroundServiceType="location" ... >  
    <!-- Any inner elements would go here. -->  
</service>
```

Prática: Permissões

- **Localização em segundo plano**
 - No Android 10 (nível 29 da API) e versões mais recentes, é necessário declarar a permissão no manifesto do App:
 - [ACCESS_BACKGROUND_LOCATION](#)

```
<manifest ... >
  <!-- Required only when requesting background location access on
        Android 10 (API level 29) and higher. -->
  <uses-permission android:name="android.permission.ACCESS_BACKGROUND_LOCATION" />
</manifest>
```

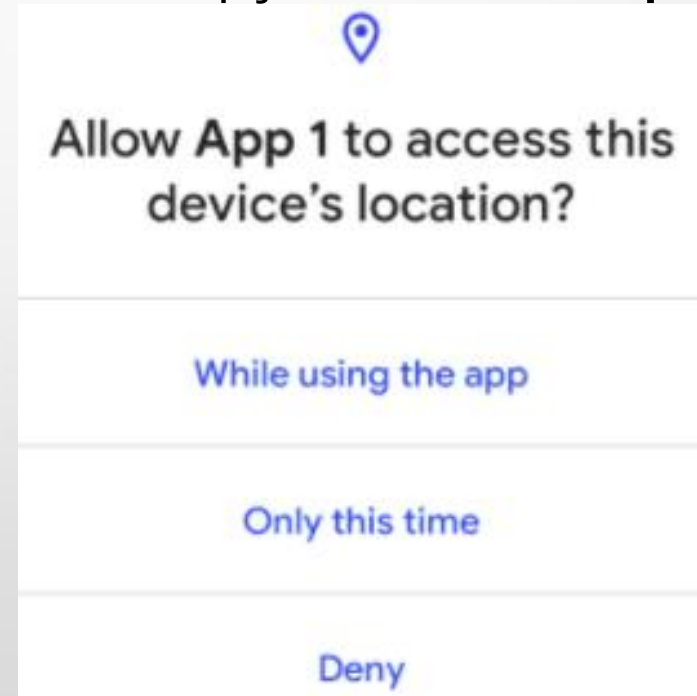
- Nas versões anteriores do Android, quando o app recebe acesso à localização em primeiro plano, também recebe acesso à localização em segundo plano automaticamente.

Prática: Permissões

- **Permissão únicas:**

- A partir do Android 11 (API de nível 30), sempre que seu app solicita uma permissão relacionada a localização, microfone ou câmera, a caixa de diálogo de permissões voltada ao usuário contém uma opção chamada **Apenas esta vez**.

- **"Permissão Temporária"**



Referências

- GLAUBER N.. Dominando o Android do Básico ao Avançado. 2. ed. São Paulo: Novatec, 2015. 950p.
- <http://developer.android.com/about/dashboards/index.html>
- DEITEL, P. J.; DEITEL, H. M.. Android para programadores: uma abordagem baseada em aplicativos. 1. ed. Porto Alegre: Bookman, 2012. 512p.
- FELKER, D.. Android application development for dummies. 1. ed. John Wiley & Sons, 2012. 456p.
- NUDELMAN, G.. Android design patterns: interaction designs. 1. ed. John Wiley & Sons, 2012. 384p.
- DARWIN, I. F.. Android cookbook. 1. ed. São Paulo: Novatec, 2012. 672p.